- Registrar ficha básica de prueba
  - Complejidad · validaciones y QR (5)

FILMINA (última) 07/10

TDD (testing ágil)

- Desarrollo dirigido por pruebas
- Es una técnica para desarrollar un software. Para entender lo que tengo que hacer, primero escribo las ~~tareas~~ pruebas (test unit.) que tienen que pasar y dsp desarrollo

Desarrollo tradicional

diseño → desarrollo → prueba

Desarrollo ágil

diseño → prueba → desarrollo

- **ATDD**: en los users, tenemos ATDD en las pruebas de aceptación de usuario

hay + tipos de pruebas (de integración, de aceptación de usuario, etc)

TDD
1) Desarrollo pruebas unitarias
2) Desarrollo código
3) Refactorizo código + limpio, reutilización

Beneficios
- código + robusto (escribo la prueba para que falle y desarrollo el código para que no falle)
- Obliga a diseñar las API (me ayuda a pensar como voy a usar ese API aunque no sepa como funciona)
- Promueve el desarrollo de componentes con alta cohesión y bajo acop.

Clave del TDD

RED: hacer test falle
GREEN: hacer test pase
REFACTOR: limpio código y reviso pruebas (elimino duplicidades)

(ver pasos último TDD)

REFACTOR: no cambia el objetivo, la funcionalidad. Lo que hace es simplificar el código, que sea más entendible y mantenible

Level

1) No escribir código hasta que no haya escrito un test case que falle

2) No escribir + tests de los necesarios. En general, un test que falla es suficiente por lo menos 1 que pase y 1 que falle por c/criterio

3) No escribir + código que el necesario para que el test pase

Patrones TDD

- Red bar → one step
  - explanation
- Green bar → Fake it
  - triangulación

hay muchos más

PRÁCTICO

Poner nombre representativo al test !!

1) Establecer pre condiciones: estado inicial necesario para ejecutar correctamente los pasos de la prueba

2) Pasos del caso de prueba: reflejan interacciones / intervenciones del usuario con el sistema y realizan el llamado a la user (función) — para un caso particular

3) Resultados: define qué debe devolver el sistema y valida que el comportamiento es correcto. Verifica mensaje final

assert → verifica condiciones. Si la cond falla dentro del assert, la prueba saltará

4) Mensaje final

En un inicio las pruebas caen (RED) cuando empecemos a escribir las funciones de c/caso de prueba de la user, las pruebas empezarán a pasar (GREEN) post refactoring

¡! Hago test unitarios, prueban una sola funcionalidad. Ver los criterios. Ademas, por cada una, tengo un test que pasa y uno que falla
↳ Devuelve un msj de xq da error

Hay algunas pruebas que por como lo implementemos, nunca van a suceder, como que falle el envío de mail, pero igual debe estar la prueba
(por mail invalido

14/10

TESTING

Objetivo: encontrar defectos } control de calidad

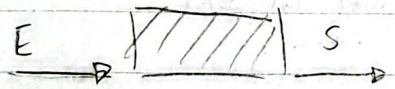
Testing agil: prevenir

[1) Plan de pruebas: cuando y cómo (casos de prueba) Buscamos la mayor cobertura de pruebas con el menor gasto/esfuerzo posible

OBJ ESTRAT [Es decir que busco la menor cant de casos de pruebas con los cuales tenga la mayor cobertura posible]

Estrategias

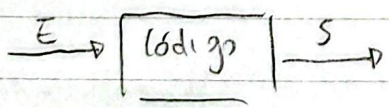
1) Caja negra ¡!



Defino el conj de entradas que van a cubrir el conj de salidas deseadas

EN TIEMPO DE EJECUCION

2) Caja blanca



Defino el conj de casos de pruebas que pasen por todas las lineas de código

ESTATICO